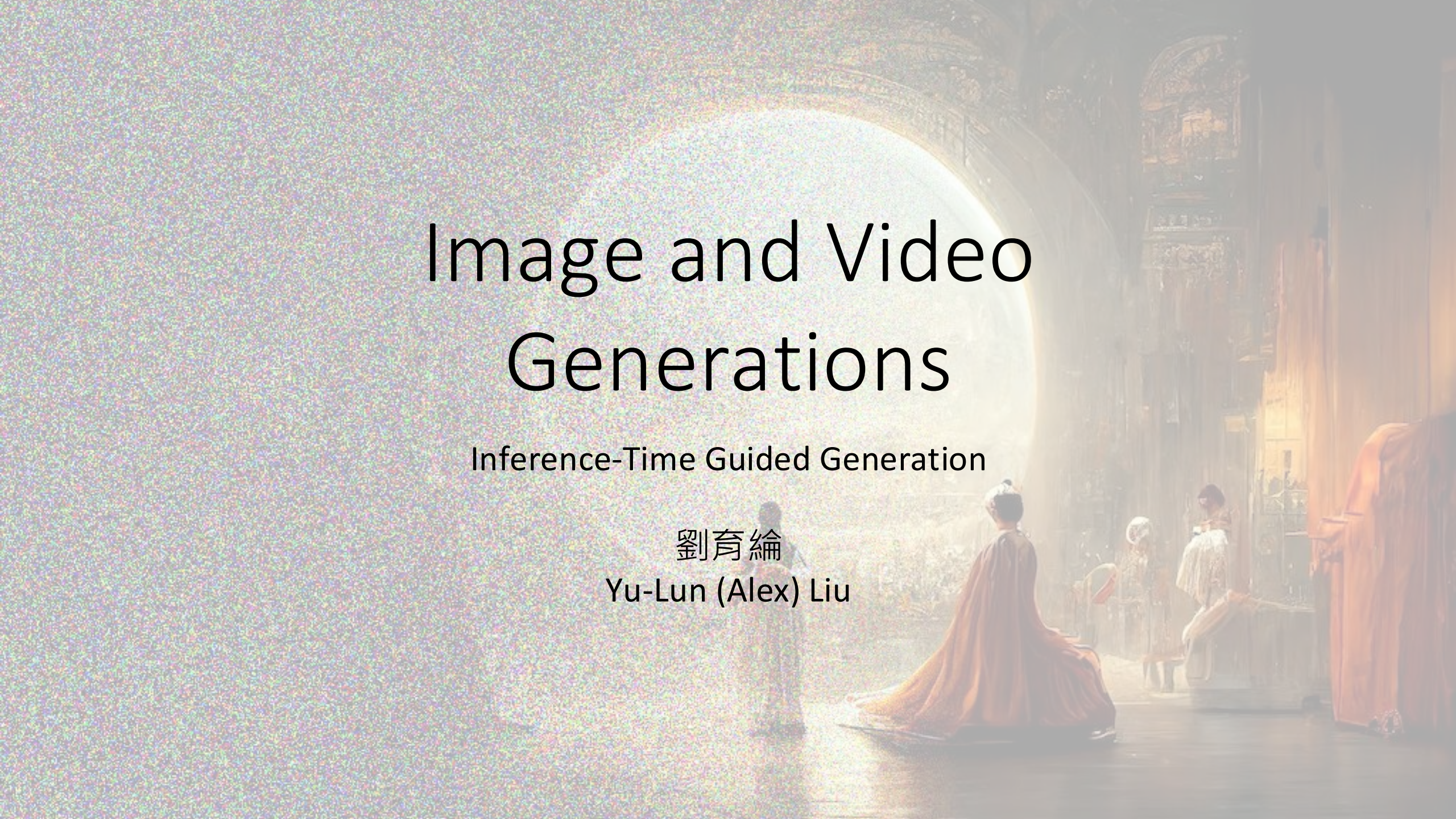


Image and Video Generations

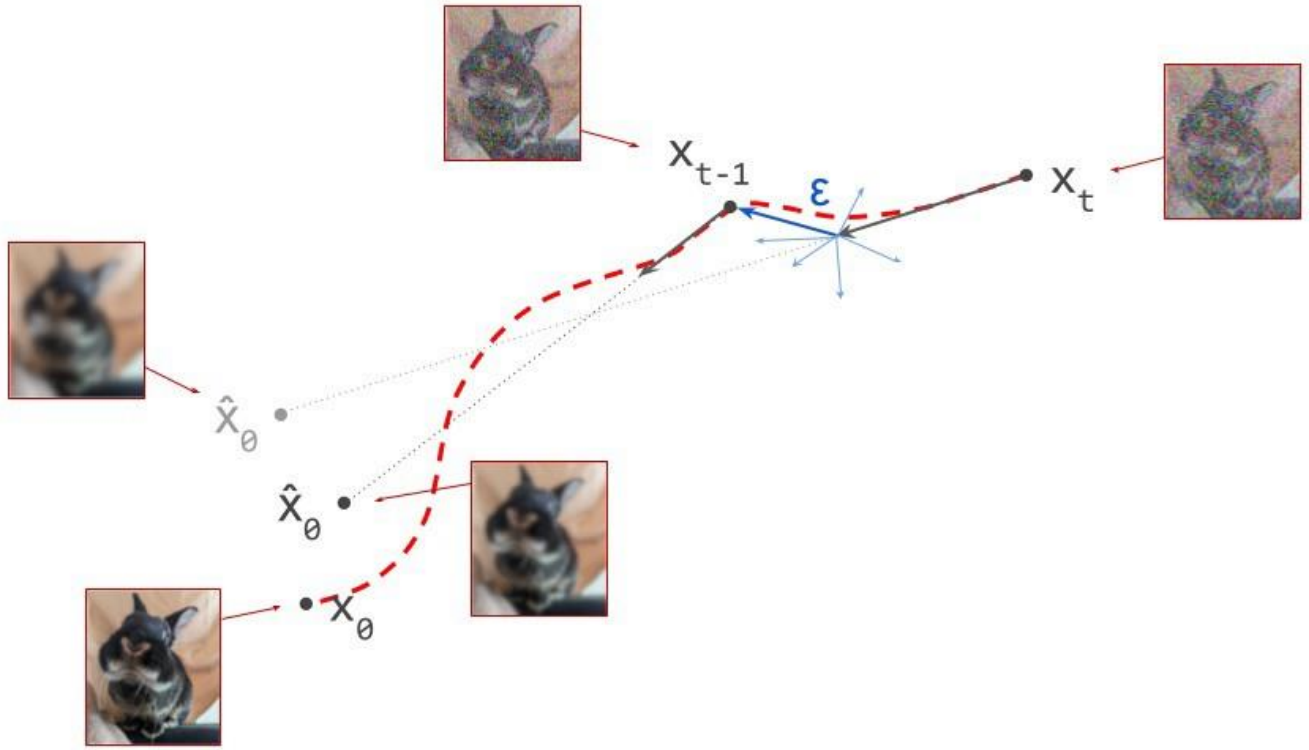
Inference-Time Guided Generation

劉育綸
Yu-Lun (Alex) Liu

https://sander.ai/2024/02/28/paradox.html

Guided Generation

The iterative generative process makes it easy to incorporate guidance.



Dhariwal and Nichol, Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021.

Recall: Classifier Guidance

Given the condition $\boldsymbol{y}$, compute the score $\nabla_{x_t} \log p(\boldsymbol{x}_t, \boldsymbol{y})$:

$$\nabla_{x_t} \log p(\boldsymbol{x}_t, \boldsymbol{y}) = \nabla_{x_t} \log p(\boldsymbol{x}_t) + \nabla_{x_t} \log p(\boldsymbol{y} \mid \boldsymbol{x}_t)$$

$$= -\frac{1}{\sqrt{1-\overline{\alpha}_t}} \hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, t) + \nabla_{x_t} \log p(\boldsymbol{y} \mid \boldsymbol{x}_t)$$

$$= -\frac{1}{\sqrt{1-\overline{\alpha}_t}} \left(\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, t) - \sqrt{1-\overline{\alpha}_t} \nabla_{x_t} \log p(\boldsymbol{y} \mid \boldsymbol{x}_t) \right)$$

How to incorporate guidance into pretrained diffusion models without further training?

Inference-Time Guided Generation

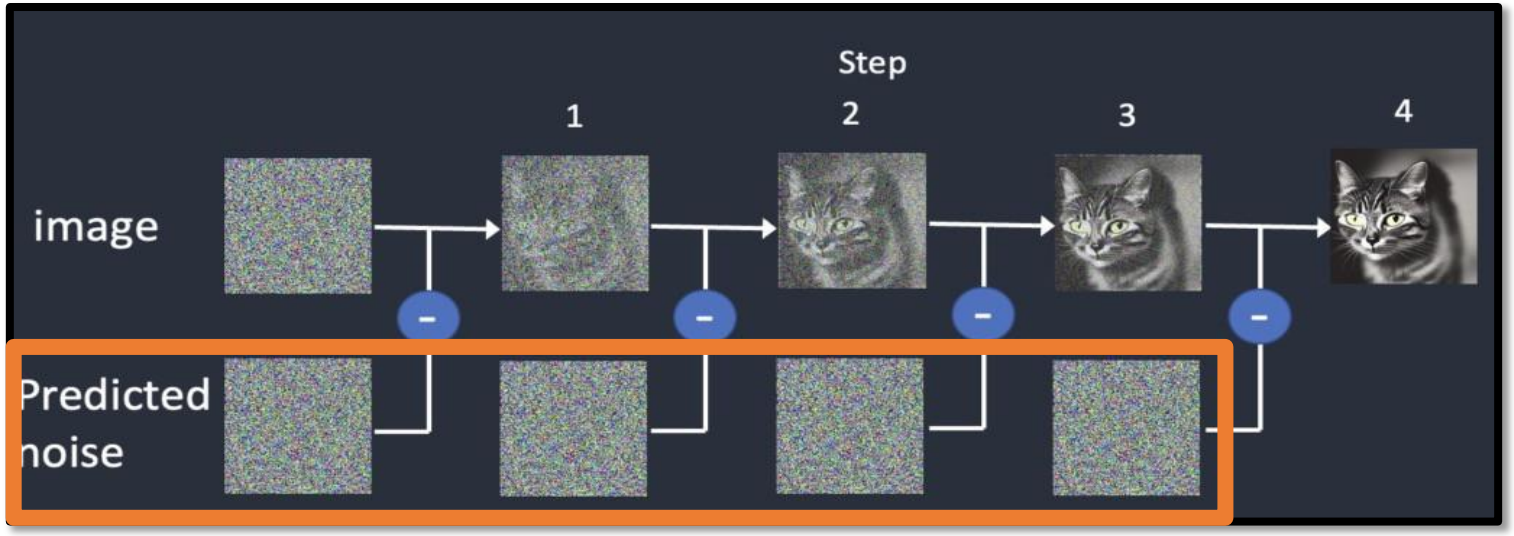
Three Categories:

1. Noise/Attention Manipulation
2. Gradient Ascent from Future Reward
3. Particle Filtering

https://stable-diffusion-art.com/how-stable-diffusion-work/

Score Manipulation

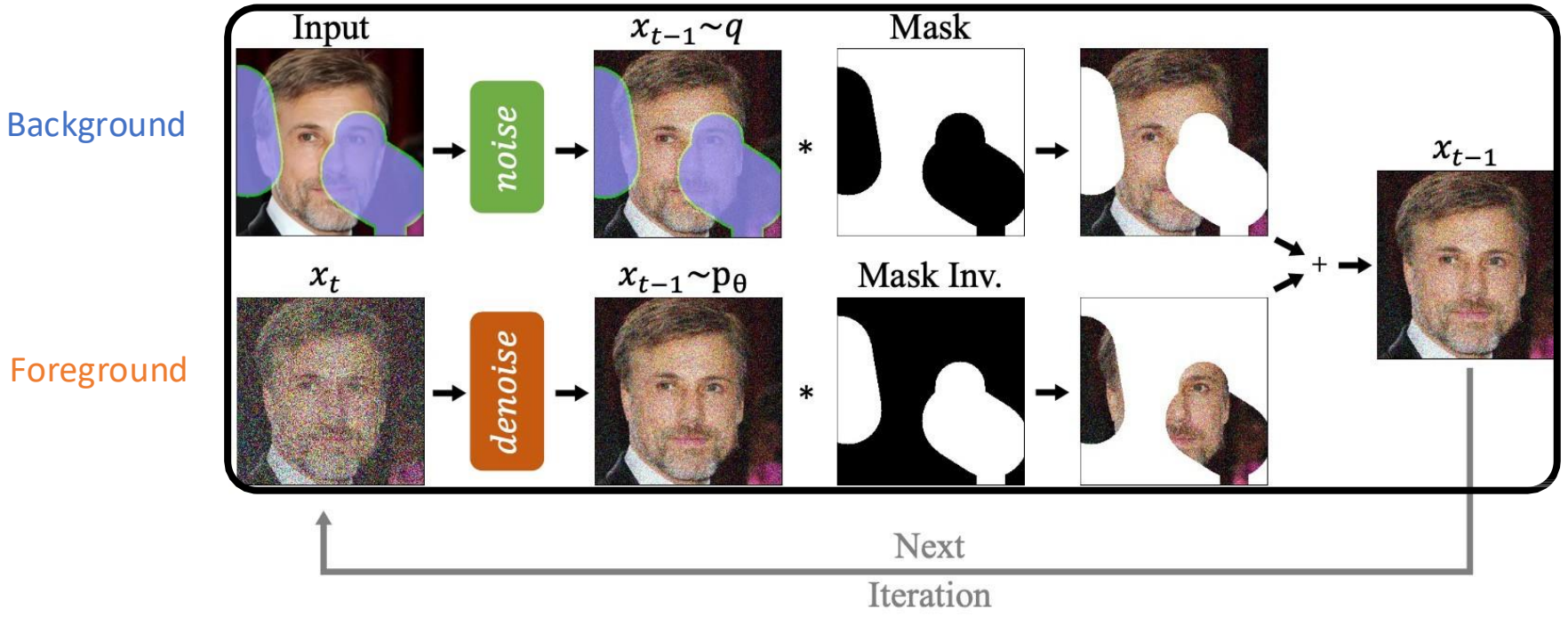
- At each step of the reverse process, the score $\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t)$ is predicted from the noisy data point $\boldsymbol{x}_t$.
- Manipulate the predicted score at each step to provide guidance.



RePaint [Lugmayr et al., 2022]

4. Repeat this process for t $= T, \ldots, 1$.

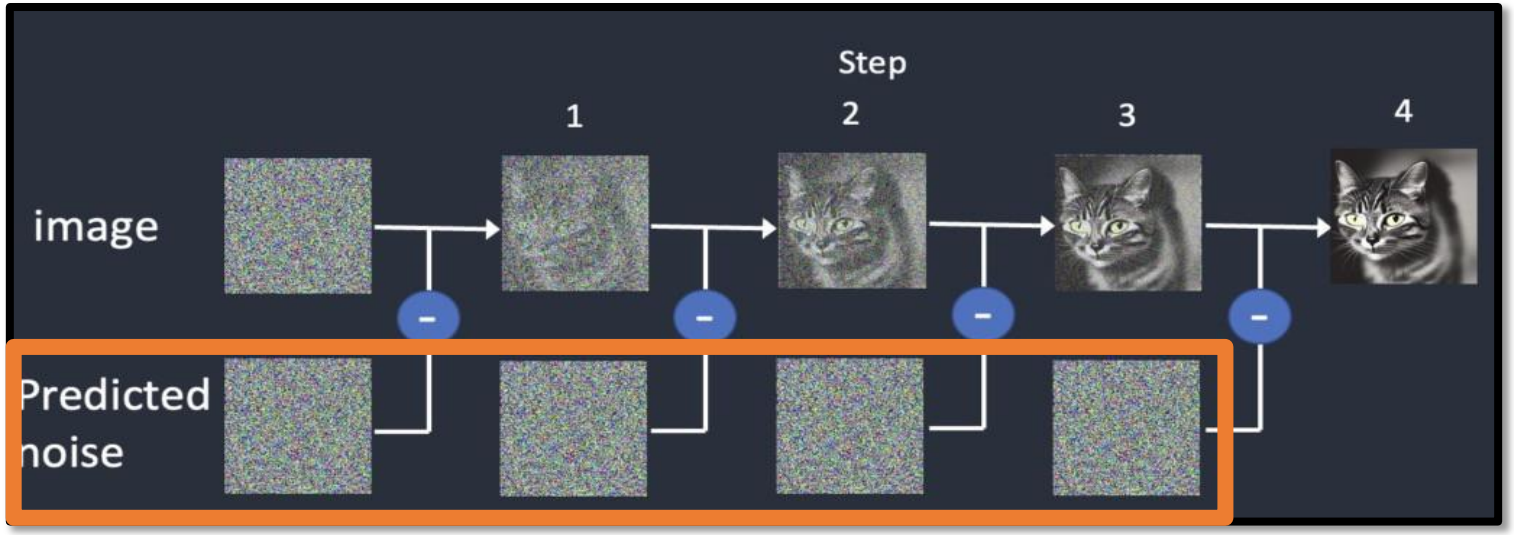
(You may need to replay the forward/reverse processes in intermediate intervals.)



https://stable-diffusion-art.com/how-stable-diffusion-work/

Attention Manipulation

- The score prediction network incorporates attention modules.
- Manipulate the attention maps in the neural network at each step to provide guidance.



Alaluf et al., Cross-Image Attention for Zero-Shot Appearance Transfer, SIGGRAPH 2024.

Appearance Transfer [Alaluf et al., 2024]



Structure



Appearance



Output

Inference-Time Guided Generation

Three Categories:

1. **Noise/Attention Manipulation**

   (+) Direct and strong guidance

   (−) Only applicable to specific spatial guidance for images

2. Gradient Ascent from Future Reward

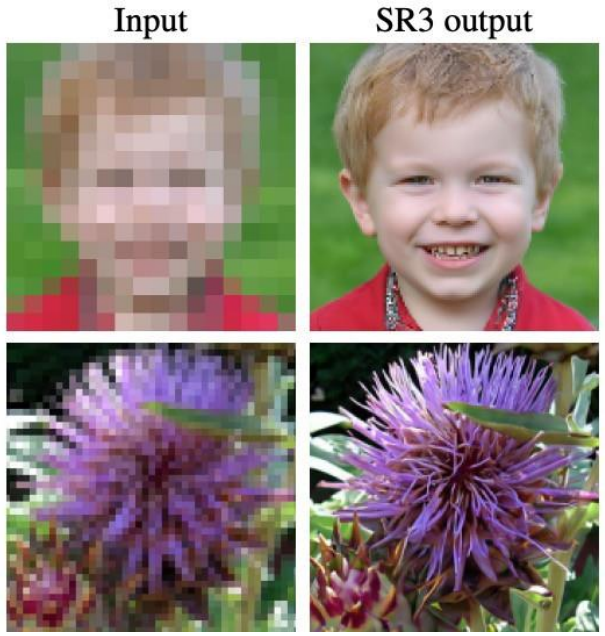
3. Particle Filtering

Inverse Problems

Gaussian Deblurring



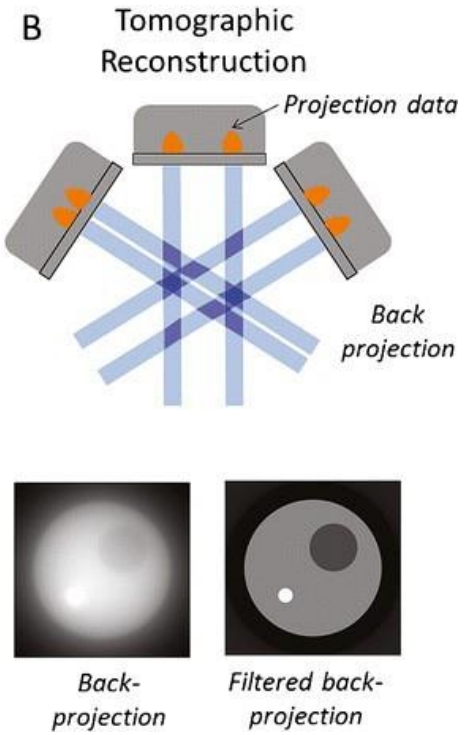
Super-Resolution



Inpainting/Outpainting



Tomography



How to model $\nabla_{x_t} \log p(y \mid x_t)$

without training?

Pseudoinverse-Guided Diffusion Models (ΠGDM)

Given $\boldsymbol{x}_t$, we can compute the expected value of $\boldsymbol{x}_0$: $\boldsymbol{x}_{0|t} = E[\boldsymbol{x}_0 \mid \boldsymbol{x}_t]$, using Tweedie's formula:

$$\boldsymbol{x}_{0|t} = E[\boldsymbol{x}_0 \mid \boldsymbol{x}_t] = \frac{1}{\sqrt{\overline{\alpha}_t}}\left(\boldsymbol{x}_t - \sqrt{1-\overline{\alpha}_t}\,\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, t)\right)$$

For simplicity, given $\boldsymbol{x}_t$, let's approximate the posterior as a Gaussian:

$$p(\boldsymbol{x}_0 \mid \boldsymbol{x}_t) \approx \mathcal{N}(\boldsymbol{x}_{0|t}, r^2 I)$$

Pseudoinverse-Guided Diffusion Models (ΠGDM)

$$p(\boldsymbol{y} \mid \boldsymbol{x}_t) = \mathcal{N}(A\boldsymbol{x}_{0|t}, \Sigma),$$
$$\Sigma = \sigma^2 I + r^2 A A^\top$$

$$\nabla_{\boldsymbol{x}_t} \log p(\boldsymbol{y} \mid \boldsymbol{x}_t) = \frac{\partial \log p(\boldsymbol{y} \mid \boldsymbol{x}_t)}{\partial \boldsymbol{x}_{0|t}} \cdot \frac{\partial \boldsymbol{x}_{0|t}}{\partial \boldsymbol{x}_t}$$

$$\approx \frac{\partial}{\partial \boldsymbol{x}_{0|t}} \left(-\frac{1}{2} (\boldsymbol{y} - A\boldsymbol{x}_{0|t})^\top \Sigma^{-1} (\boldsymbol{y} - A\boldsymbol{x}_{0|t}) \right) \cdot \frac{\partial \boldsymbol{x}_{0|t}}{\partial \boldsymbol{x}_t}$$

$$= A^\top \Sigma^{-1} (\boldsymbol{y} - A\boldsymbol{x}_{0|t}) \cdot \frac{\partial \boldsymbol{x}_{0|t}}{\partial \boldsymbol{x}_t}$$

$$= A^\top (\sigma^2 I + r^2 A A^\top)^{-1} (\boldsymbol{y} - A\boldsymbol{x}_{0|t}) \cdot \frac{\partial \boldsymbol{x}_{0|t}}{\partial \boldsymbol{x}_t}$$

Chung et al., Diffusion Posterior Sampling for General Noisy Inverse Problems, ICLR 2023.

Diffusion Posterior Sampling (DPS)

Now we do not need to assume a linear function:

$$p(\boldsymbol{y} \mid \boldsymbol{x}_t) \approx \mathcal{N}(\mathcal{A}(\boldsymbol{x}_{0|t}), \sigma^2 I)$$

The conditional score $\nabla_{x_t} \log p(\boldsymbol{y} \mid \boldsymbol{x}_t)$ can now be computed in a much simpler form:

$$\nabla_{x_t} \log p(\boldsymbol{y} \mid \boldsymbol{x}_t) \approx \nabla_{x_t} \left(-\frac{1}{2\sigma^2} \left\| \boldsymbol{y} - \mathcal{A}(\boldsymbol{x}_{0|t}) \right\|^2 \right)$$

$$= -\frac{1}{2\sigma^2} \nabla_{x_t} \left\| \boldsymbol{y} - \mathcal{A}(\boldsymbol{x}_{0|t}) \right\|^2$$

Chung et al., Diffusion Posterior Sampling for General Noisy Inverse Problems, ICLR 2023.

Diffusion Posterior Sampling (DPS)

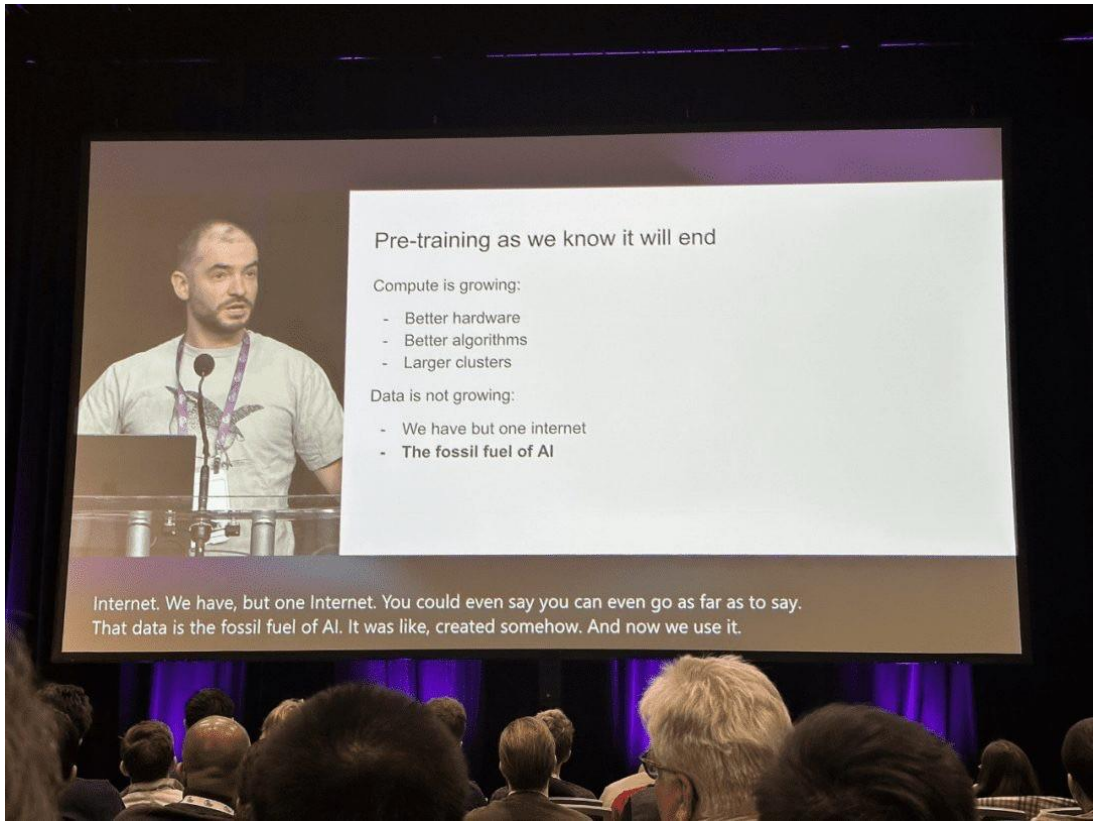
$$q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}\left(\frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}\boldsymbol{x}_t + \frac{\sqrt{\overline{\alpha}_{t-1}}\beta_t}{1-\overline{\alpha}_t}\boldsymbol{x}_0,\ \widetilde{\sigma}_t^2 I\right)$$

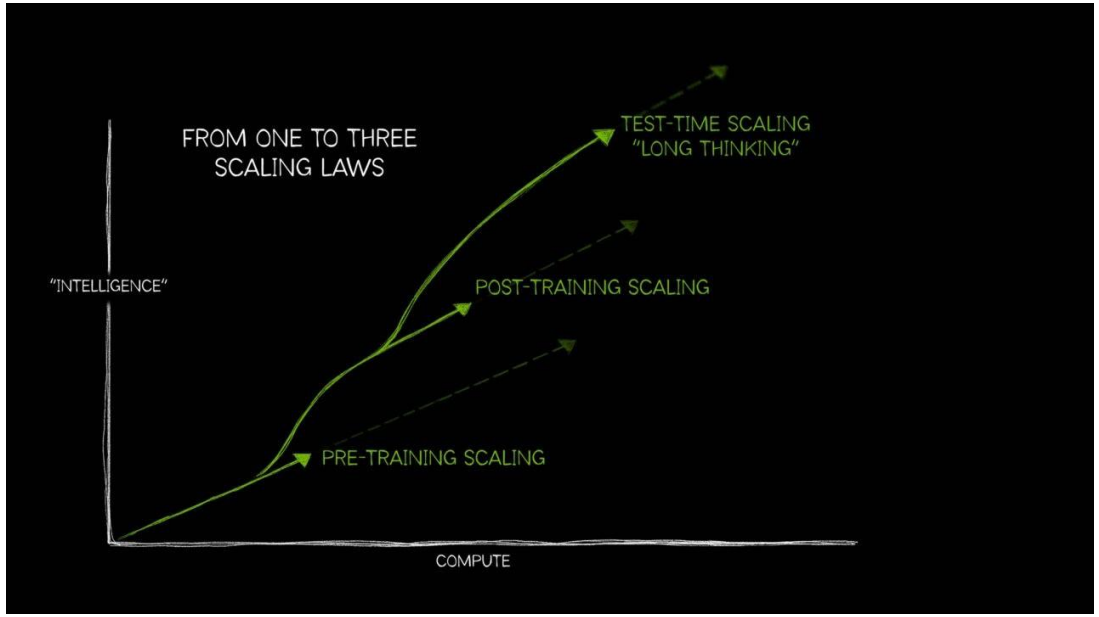
For each time step $t = T, \ldots, 1$, repeat:

1. $\boldsymbol{x}_{0|t} = \frac{1}{\sqrt{\overline{\alpha}_t}}\left(\boldsymbol{x}_t - \sqrt{1-\overline{\alpha}_t}\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, t)\right)$
2. $\widetilde{\mu} = \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}\boldsymbol{x}_t + \frac{\sqrt{\overline{\alpha}_{t-1}}\beta_t}{1-\overline{\alpha}_t}\boldsymbol{x}_{0|t}$
3. $\boldsymbol{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
4. $\boldsymbol{x}'_{t-1} = \widetilde{\mu} + \widetilde{\sigma}_t \boldsymbol{z}_t$
5. $\boldsymbol{x}_{t-1} = \boldsymbol{x}'_{t-1} - \xi_t \nabla_{\boldsymbol{x}_t}\left\|\boldsymbol{y} - \mathcal{A}(\boldsymbol{x}_{0|t})\right\|^2$ ← *One line of code!*

https://www.aisharenet.com/en/ilya-sutskeverzaineu/
https://blogs.nvidia.com/blog/ai-scaling-laws/

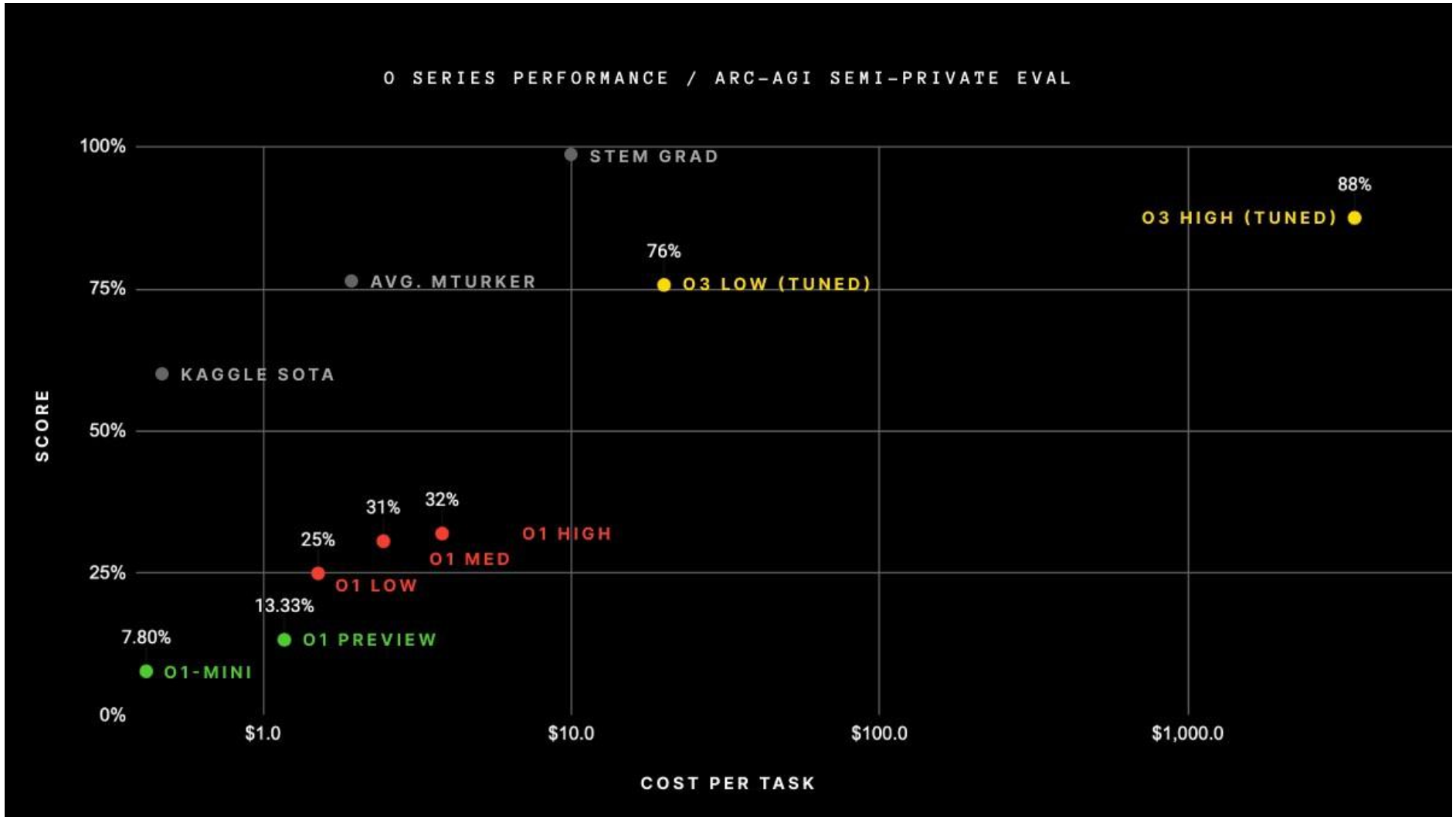
Off-Topic: The End of the Pre-Training Era





https://arcprize.org/blog/oai-o3-pub-breakthrough

Inference-Time Scaling – OpenAI o3



Google DeepMind 2024-8-7

Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

Charlie Snell[♦, 1], Jaehoon Lee[2], Kelvin Xu[♣, 2] and Aviral Kumar[♣, 2]
[♣]Equal advising, [1]UC Berkeley, [2]Google DeepMind, [♦]Work done during an internship at Google DeepMind

2. A Unified Perspective on Test-Time Computation: Proposer and Verifier

We first unify approaches for using test-time computation and then analyze some representative methods. First, we view the use of additional test-time compute through the lens of modifying the model's predicted distribution *adaptively* at test-time, conditioned on a given prompt. Ideally, test-time compute should modify the distribution so as to generate better outputs than naïvely sampling from the LLM itself would. In general, there are two knobs to induce modifications to an LLM's distribution: **(1) at the input level**:

computation: (1) searching against dense, process-based verifier reward models; and (2) updating the model's distribution over a response adaptively, given the prompt at test time. We find that in both cases, the effectiveness of different approaches to scaling test-time compute critically varies depending on the difficulty of the prompt. This observation motivates applying a "compute-optimal" scaling strategy, which acts to most effectively allocate test-time compute adaptively per prompt. Using this compute-optimal strategy, we can improve the efficiency of test-time compute scaling by more than 4× compared to a best-of-N baseline. Additionally, in a FLOPs-matched evaluation, we find that on problems where a smaller base model attains somewhat non-trivial success rates, test-time compute can be used to outperform a 14× larger model.

Gradient Ascent from Future Reward

When the user preference is modeled as
a reward function $\mathcal{R}$,
generate a realistic data point $\boldsymbol{x}_0 \sim p_0$ that
yields a high reward.

*Peters and Schaal, Reinforcement learning by reward-weighted regression for operational space control, ICML 2007.

Reward-Guided Generation

Given a reward function $\mathcal{R}$, the goal is to sample from the following modified data distribution p_0^* (instead of p_0):

$$p_0^* = \overbrace{\underset{q}{\operatorname{argmax}}\, \mathbb{E}_{x_0 \sim q}[\mathcal{R}(\boldsymbol{x}_0)]}^{\text{Reward Maximization}} - \lambda \overbrace{\mathcal{D}_{\mathrm{KL}}[q \parallel p_0]}^{\text{KL Regularization}}$$

which can be computed as[*]

$$p_0^*(\boldsymbol{x}_0) = \frac{1}{Z_0} p_0(\boldsymbol{x}_0) e^{\frac{\mathcal{R}(\boldsymbol{x}_0)}{\lambda}}$$

Same Objective as RL-Based Finetuning

The objective function for the target distribution optimization is the same as the one for RL-based finetuning methods for pretrained models.

E.g., Direct Preference Optimization (DPO)

[Rafailov et al., NeurIPS 2023]

RL Fine-Tuning Phase: During the RL phase, the learned reward function is used to provide feedback to the language model. Following prior works [17, 18], the optimization is formulated as

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right] - \beta\mathbb{D}_{\mathrm{KL}}\left[\pi_\theta(y \mid x) \,||\, \pi_{\mathrm{ref}}(y \mid x)\right], \tag{3}$$

Optimal Value Function

The distribution for intermediate sample $\boldsymbol{x}_t$ is also formulated as follows:

$$p_t^*(\boldsymbol{x}_t) = \frac{1}{Z_t} p_t(\boldsymbol{x}_t) e^{\frac{\mathcal{V}(\boldsymbol{x}_t)}{\lambda}}$$

where $\mathcal{V}$ is the optimal value function. Let's simply approximate it using the expected future output $\boldsymbol{x}_{0|t}$:

$$\mathcal{V}(\boldsymbol{x}_t) \approx \mathcal{R}(\boldsymbol{x}_{0|t})$$

Score for the Modified Distribution

When the reward function $\mathcal{R}$ is differentiable, the score for the modified distribution $p_t^*(\boldsymbol{x}_t)$ is computed by taking the gradient of the log-likelihood:

$$\nabla_{\boldsymbol{x}_t} \log p_t^*(\boldsymbol{x}_t) \approx \nabla_{\boldsymbol{x}_t} \log \left[p_t(\boldsymbol{x}_t) e^{\frac{\mathcal{R}(\boldsymbol{x}_{0|t})}{\lambda}} \right]$$

$$= \overbrace{\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t)}^{\text{Original Score}} + \frac{1}{\lambda} \overbrace{\nabla_{\boldsymbol{x}_t} \mathcal{R}(\boldsymbol{x}_{0|t})}^{\text{Gradient of Expected Future Reward}}$$

Chung et al., Diffusion Posterior Sampling for General Noisy Inverse Problems, ICLR 2023.

Diffusion Posterior Sampling (DPS)

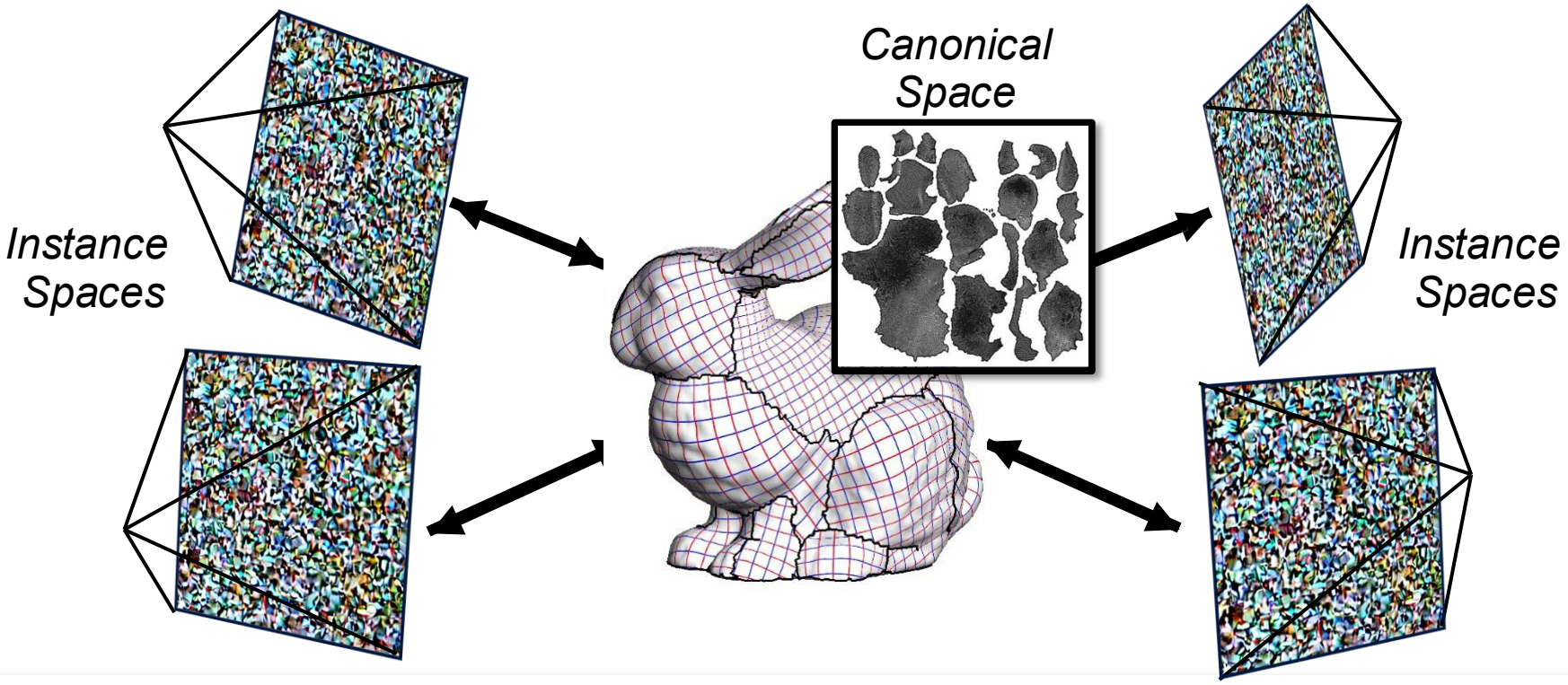
1. Compute $\boldsymbol{x}_{t-\Delta t} \leftarrow \boldsymbol{x}_t - h(t, \boldsymbol{x}_t)\Delta t + g(t)\sqrt{\Delta t} \cdot \varepsilon.$

2. Update $\boldsymbol{x}_{t-\Delta t} = \boldsymbol{x}_{t-\Delta t} + \xi_t \Delta t \nabla_{\boldsymbol{x}_t} \mathcal{R}\left(\boldsymbol{x}_{0|t}\right).$

This update is exactly equivalent to DPS, interpreted through the lens of an RL problem.

Extension: Diffusion Synchronization

The gradient ascent approach can be used to synchronize multiple generative processes.



Applications: Panorama Generation

- **Canonical** space: Panorama latent space
- **Instance** space: Square latent space



Inference-Time Guided Generation

Three Categories:

1. Noise/Attention Manipulation
2. **Gradient Ascent from Future Reward**

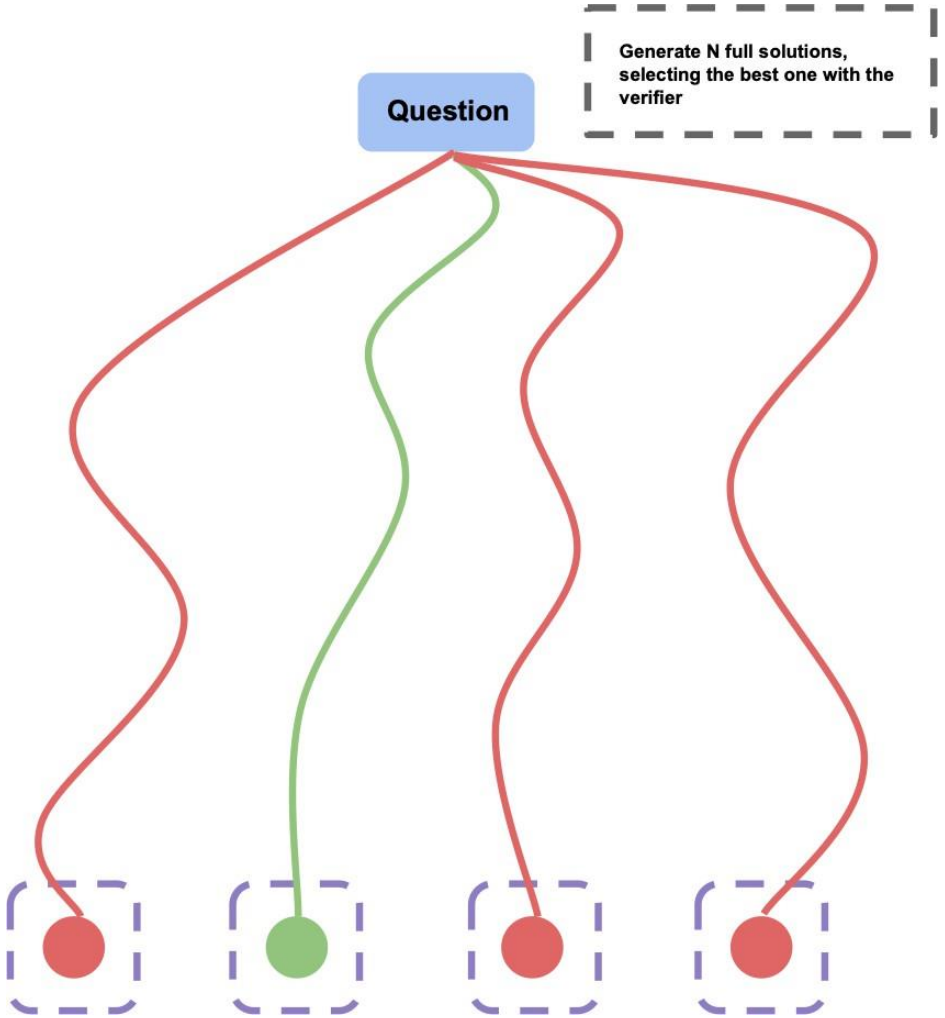
   (+) Applicable to a wider range of applications.

   (–) Still limited to cases where the reward is differentiable.

3. Particle Filtering

Snell et al., Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters, arXiv 2024.

Best-of-N



https://annchenn.github.io/Manga2Animate/

Applications: Manga -> Animate





Snell et al., Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters, arXiv 2024.

Process Reward for Intermediate Results

Optimizing the verifier. In our abstraction of the proposal distribution and verifier, the verifier is used to aggregate or select the best answer from the proposal distribution. The most canonical way to use such a verifier is by applying best-of-N sampling, wherein we sample N complete solutions and then select the best one according to a verifier [7]. However, this approach can be further improved by training a process-based verifier [22], or a process reward model (PRM), which produces a prediction of the correctness of each intermediate step in an solution, rather than just the final answer. We can then utilize these per-step predictions to perform tree search over the space of solutions, enabling a potentially more efficient and effective way to search against a verifier, compared to naïve best-of-N [6, 10, 48].

Intermediate Results in Diffusion Models

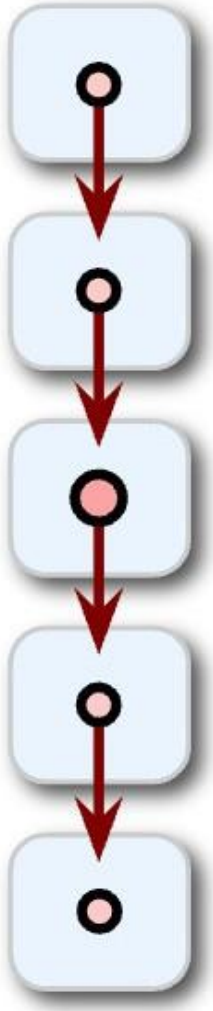
- In diffusion and flow models, intermediate outputs are noisy data points from which the expected final outputs can be computed.

- Final output rewards can serve as process rewards!



Particle Filtering for Diffusion Models

- The denoising process of a diffusion model can be viewed as a Sequential Monte Carlo process with a single trajectory.
- Only one particle is sampled at each step.



Arnaud Doucet, Sequential Monte Carlo Methods in Practice, 2001.

Sequential Monte Carlo (SMC)

- How to sample from $p_\theta^*(\boldsymbol{x}_{t-\Delta t}|\boldsymbol{x}_t)$?
  (Note that the normalizer Z_t' is not tractable.)

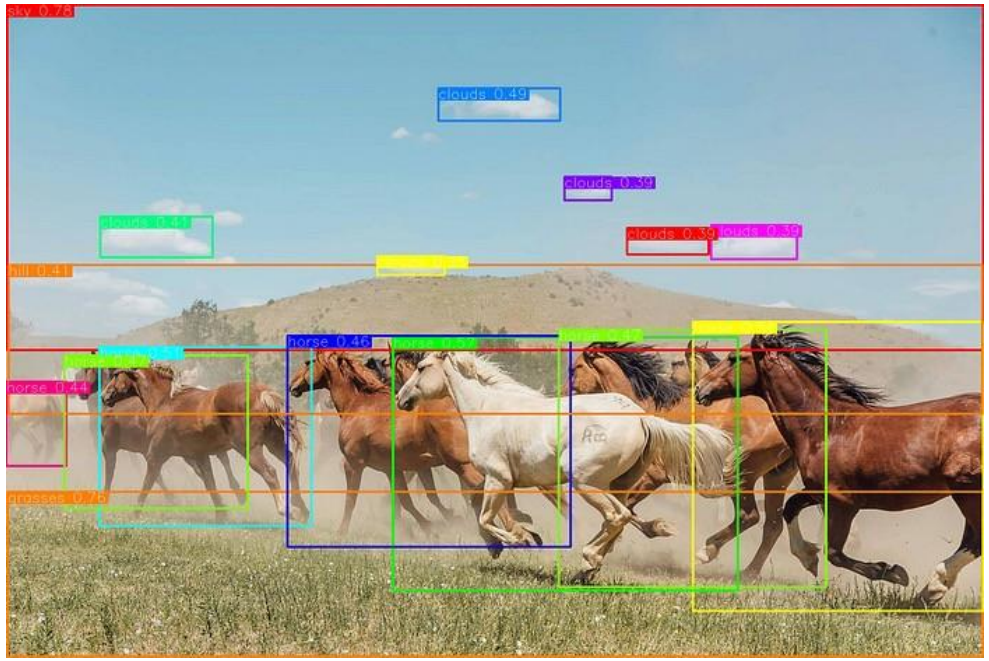
- **Sequential Monte Carlo (SMC) / Particle Filtering**:
  Approximate sampling from $p_\theta^*(\boldsymbol{x}_{t-\Delta t}|\boldsymbol{x}_t)$ by drawing multiple samples $\boldsymbol{x}_{t-\Delta t}^{(i)} \sim p_\theta(\boldsymbol{x}_{t-\Delta t}|\boldsymbol{x}_t)$ and applying importance sampling and resampling.

Liu et al., Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection.

Sequential Monte Carlo (SMC)

Applications:

Any case where a non-differentiable reward function is given. E.g., Object count.



Chung et al., Diffusion Posterior Sampling for General Noisy Inverse Problems, ICLR 2023.

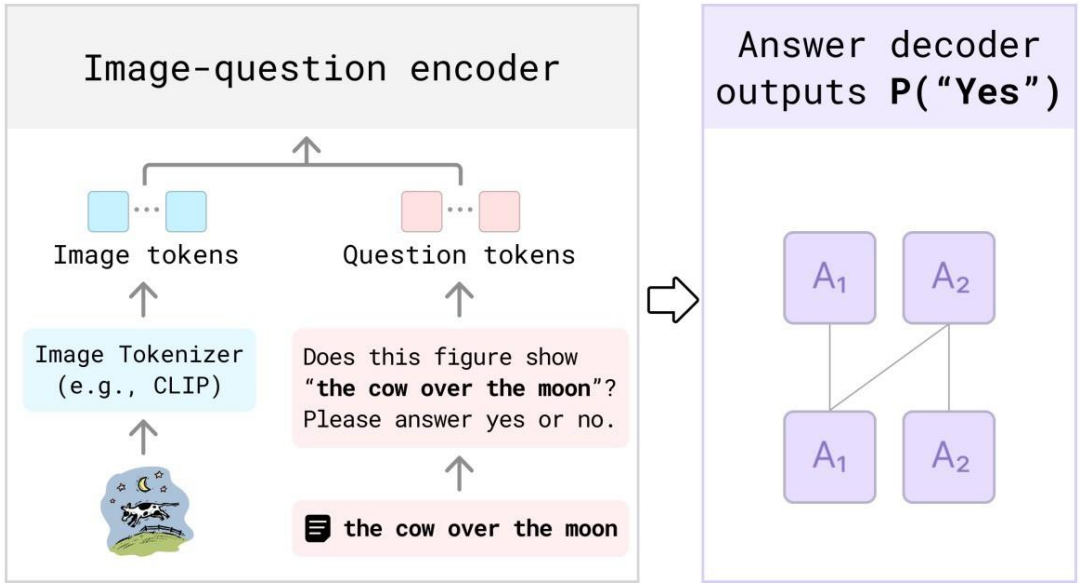
Sequential Monte Carlo (SMC)

Applications:

Any case where a discriminative model is a **closed model**

from which the gradient cannot be accessed.

E.g., Precise text alignment using a closed VLM.



Goal

GPT 4o



FLUX + Inference-Time Scaling



“Four drums, seven tomatoes, and five candles.”

Inference-Time Guided Generation

Three Categories:

1. Noise/Attention Manipulation
2. Gradient Ascent from Future Reward
3. **Particle Filtering**

   (+) Applicable to both differentiable and non-differentiable rewards.

   (−) Slow.

Sequential Monte Carlo (SMC)

Q. When does it become close?

A. This happens when the expected final output $\mathbf{x}_{0|t}$ is close to the corresponding real data point.

Q. What kinds of models satisfy this?

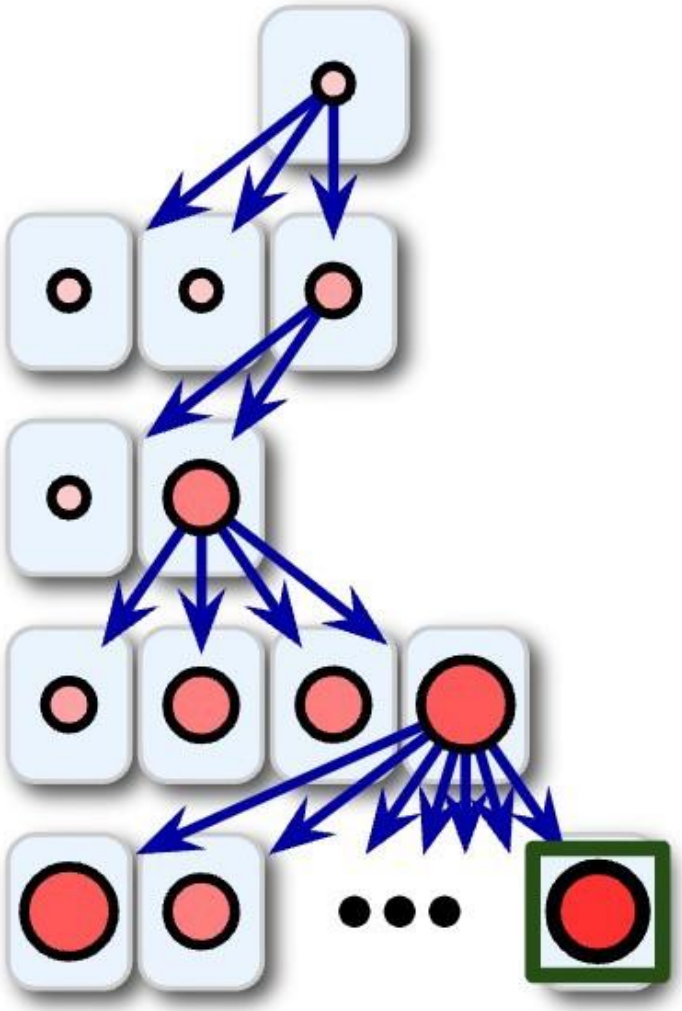
A. Few-step models such as consistency models and reflow models!

Flow Models + SMC

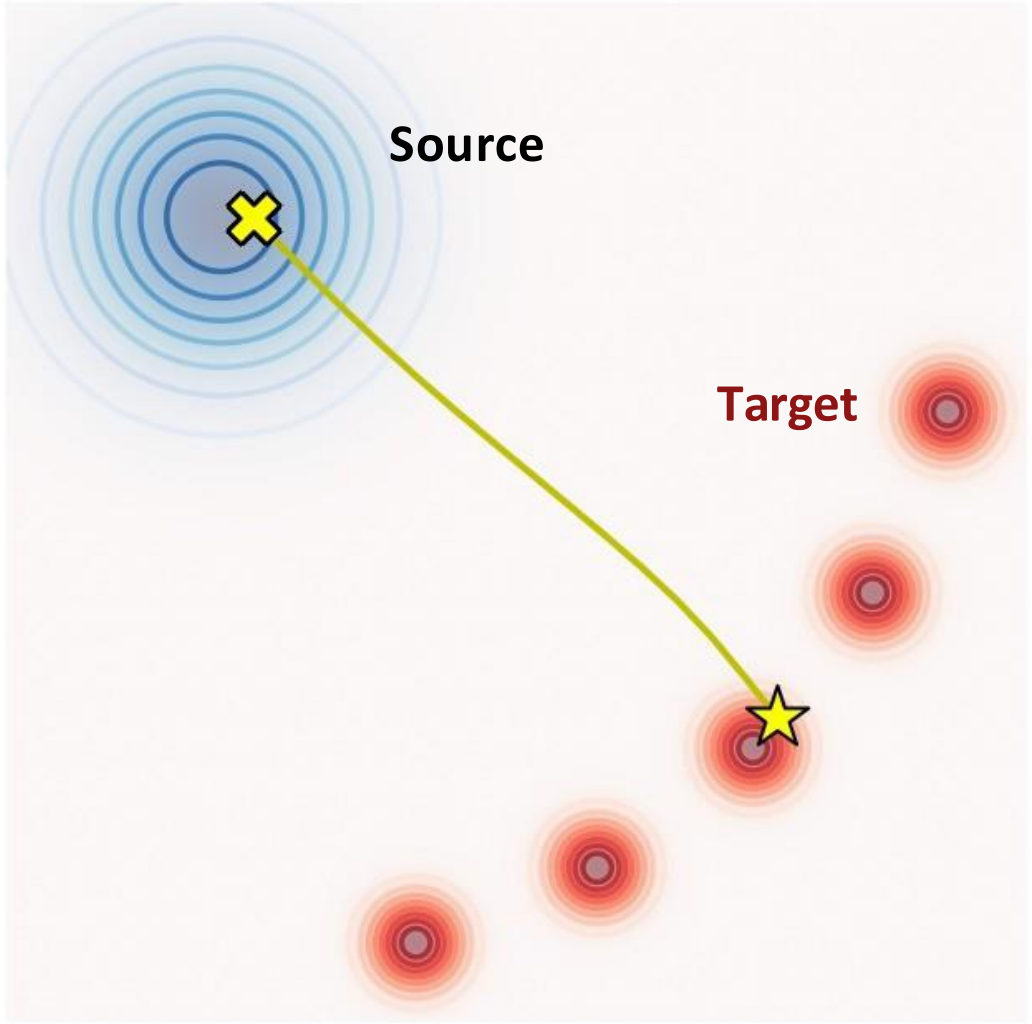
No. SMC relies on

1. **multi-step** generation and
2. **stochasticity** at each step!

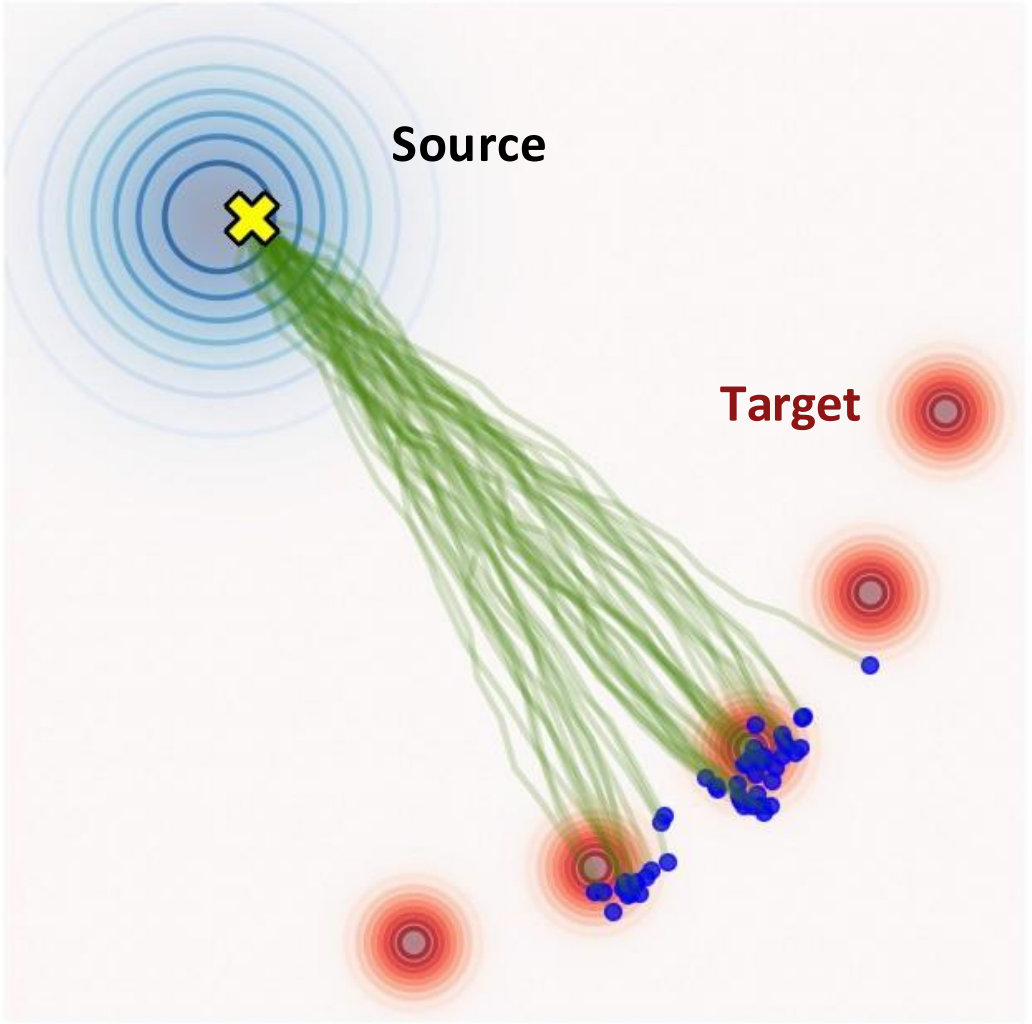
We therefore need to run a multi-step SDE-based generative process.



ODE-SDE Conversion



Linear ODE



Linear SDE

Takeaways

Inference-Time Guided Generation

Noise/Attention Manipulation

- Spatial grounding only

Gradient Ascent from Future Reward

- Differentiable reward only

Particle Filtering

- Any case where a reward function is defined.